

GLIDE

Arbeitsvorbereitung für Codex, Build, Release und Dokumentation

Windows · macOS · Branding · Packaging · QA · Signing · Stores · Automatisierung

Zweck und Abgrenzung dieses Dokuments

Diese Revision beschreibt Glide 3.2.0 mit Datenformat 10 und den noch offenen Weg zu einem öffentlichen Release. Code, Dokumentation und Beispiele sind konsolidiert; Build, Signing, Produktidentität und manuelle Plattformfreigabe bleiben offen. Historische Screenshots und Ticketvorlagen sind Referenzen.

Dokumentversion 3.2.0 · Stand 04.09.2026

Arbeitsstatus: Source 3.2.0 · Datenformat 10 · interne Vorabversion ohne freigegebenen Release-Build

Inhaltsübersicht

1. Management Summary	3	13. Reproduzierbares QA-Testsystem.....	16
2. Ausgangslage und vorhandene Unterlagen	3	14. Referenz-Testdaten und Migration.....	17
3. Was bereits richtig vorbereitet ist	5	15. Release-Artefakte und Versionierung	17
4. Ergänzungen und offene Vorbereitungspunkte	6	16. Security und Software-Supply-Chain	17
5. Branding- und Icon-Vorbereitung	7	17. Empfohlener Grafik- und Asset-Workflow	18
6. Umgesetzte Gesamt- und Ordnerstruktur.....	8	18. Nachgeholte Arbeitsvorbereitung.....	18
7. Aufgabenverteilung Codex / Benutzer	9	19. Verbindlicher Phasenplan	20
8. Empfohlene Codex-Arbeitspakete.....	10	20. Strategische Leitprinzipien.....	20
9. Eingearbeitete AGENTS.md für Glide	13	21. Zielzustand Release-Workflow.....	21
10. Eingerichtetes Dokumentationssystem.....	14	22. Fazit und nächste Schritte	21
11. Zentrales Produkt- und Identitätsregister.....	15	23. Quellen und Prüfstand	21
12. Apple-/Microsoft-Accountentscheidungen	15		

Die Kapitel sind in Prozessreihenfolge angeordnet. Die Seitenzahlen und internen Sprungziele werden aus dem finalen Render dieser Revision erzeugt.

1. Management Summary

Gesamtbewertung. Glide 3.2.0 ist eine lokale Aufgaben- und Listen-App mit vier Punktarten, verschachtelten Ordnern, Labels, Papierkorb, Kalender und optionaler Uhrzeit. Der Code bleibt ein Python-/Tk-Monolith. Datenformat 10 und portable Backups der Formate 4 bis 10 werden unterstützt. Für die Veröffentlichung fehlen insbesondere reale Plattformfreigabe, Produktidentität, Branding, Build und Signing.

Der nächste Qualitätssprung liegt weiterhin außerhalb der reinen Feature-Implementierung: vollständige manuelle Plattform-QA, verbindliche Produktidentitäten, reproduzierbares Packaging, Signing/Notarisierung und Clean-Machine-Tests. Die bestehende Struktur trennt dafür Source, Tests, Fixtures, technische Dokumentation, Grafik-Master, Store-Material und finale Release-Artefakte.

Kernaussage 3.2.0

Seit 3.1.0 bündeln item_change und sidebar_change die Änderungen, run_modal die modalen Dialoge. 3.2.0 entfernt die automatische Datenübernahme aus früheren Programmnamen und zentralisiert viele Symbole in ICONS. Gruppen-, Wichtigkeits- und Beschreibungsmarker enthalten weiterhin Emoji-Ausnahmen. Das gemeldete Einfrieren bleibt mangels Reproduktion NICHT VERIFIZIERT.

Die drei wichtigsten nächsten Arbeiten sind:

1. Produkt-, Publisher-, Lizenz- und Plattformidentitäten verbindlich festlegen.
2. Die manuelle Windows-/macOS-QA-Matrix auf realen Zielsystemen abschließen.
3. Reproduzierbare unsigned Builds vorbereiten; erst danach Signing, Notarisierung und Stores angehen.

2. Ausgangslage und vorhandene Unterlagen

2.1 Vorhandene Build- & Release-Anleitung

Die vorhandene Anleitung beschreibt bereits einen vollständigen Zielprozess von der Projektvorbereitung bis zur Veröffentlichung:

- Projekt und Stammdaten vorbereiten
- Branding und Icon-Master vorbereiten
- Windows mit PyInstaller bauen
- Windows-Installer mit Inno Setup erstellen
- Windows Code Signing
- macOS-App-Bundle erzeugen
- DMG erstellen
- Apple Developer ID, Signierung und Notarisierung
- Benutzerdaten, Einstellungen, Backups und Logs trennen
- Versionierung und Updatefähigkeit
- Clean-Machine- und Release-Tests
- Microsoft Store und Mac App Store
- spätere Automatisierung und KI-Unterstützung

Die Build- und Release-Abfolge ist eine Planung. Im Bestand fehlen noch PyInstaller-Spec, Installer-Skript, macOS-Bundle-Konfiguration und freigegebene Branding-Assets. Dieses Dokument trennt deshalb vorhandene Software von künftig herzustellenden Release-Artefakten.

2.2 Funktionskatalog und Produktphilosophie

Glide arbeitet lokal, einsprachig Deutsch und ohne Konto, Netzwerkkommunikation oder Telemetrie. Die Laufzeit verwendet nur die Python-Standardbibliothek und Tk. Cloud-Synchronisierung, Mehrbenutzerbetrieb, Konten und Lizenzserver sind bewusste Nicht-Ziele gemäß docs/01_PRODUCT_CONSTRAINTS.md.

Konsequenz für Codex

Release-, Packaging- und Strukturarbeiten dürfen diese Produktphilosophie nicht stillschweigend verändern. Neue Runtime-Abhängigkeiten, Cloud-Dienste, Telemetrie, Kontensysteme oder Lizenzserver dürfen nicht „nebenbei“ eingeführt werden.

2.3 Aktuelle Arbeitsstruktur

	__pycache__		31.08.2026 13:06	Dateiordner	
	05_Probelisten_Testdaten		21.07.2026 11:48	Dateiordner	
	07_Python-Versionen		31.08.2026 16:02	Dateiordner	
	10_Dokumentation		31.08.2026 16:02	Dateiordner	
	100_Archiv		31.08.2026 12:08	Dateiordner	
	_test_glide_v250.py		31.08.2026 13:09	Python File	8 KB

Abbildung 1: Historischer Zwischenstand während des vorherigen Laufs (31.08.2026).

Der Screenshot dokumentiert einen frühen Zwischenstand des vorherigen Laufs, nicht dessen tatsächlichen Abbruchpunkt. Zu diesem Zeitpunkt lagen Versionsdateien unter 07_Python-Versionen, der Test noch im Workspace-Root und ein generierter __pycache__ daneben. Danach wurden Source 2.5.1, Tests, Struktur und Word-Dokument bereits fertiggestellt.

2.4 Umgesetzter Arbeitsstand 3.2.0

Kanonischer Source ist 01_Repository/Glide/src/glide/app.pyw. Die aktuelle Einzeldatei unter 07_Python-Versionen ist bytegleich; ältere Stände bleiben unverändert im dortigen Archiv. Arbeitsdokumente werden vor Fortschreibung im Archiv ihres jeweiligen Ordners erhalten.

```

Glide ToDo/
├── 00_Arbeitsvorbereitung/           aktuelle Vorbereitung 3.2.0
├── 01_Repository/Glide/
│   ├── src/glide/app.pyw           kanonischer Monolith
│   ├── tests/integration/          drei ausführbare Suiten
│   ├── tests/tools/                 Prüf- und Datenwerkzeuge
│   ├── tests/fixtures/             Formate 4-10, Legacy 2, Beispiele
│   ├── docs/                       aktuelle technische Dokumentation
│   └── assets/, packaging/         Release-Arbeit noch offen
├── 05_Probelisten_Testdaten/        aktuelle Glide-Beispiele
├── 07_Python-Versionen/             aktuelle Kopie und Archiv
├── 10_Dokumentation/               Word-Arbeitsgrundlage 3.2.0
├── 20_Grafik_Master/               noch keine freigegebenen Assets
├── 30_Release_Exports/             noch kein freigegebener Build
├── 40_Store_Material/              Texte, Anforderungen, Quellen
├── 50_Ablage/                      Prüfberichte und Renderläufe
└── 90_Testdaten_Extern/            historische Migrationseingaben
Archive liegen dezentral im jeweiligen Ausgangsordner.

```

Drei Suiten prüfen Hauptfunktionen, Datenintegrität und appweite Pfade. Dazu kommen statische Analyse, Erreichbarkeitsanalyse und Sichtaufnahmen. Die Testisolierung setzt GLIDE_DATA_DIR vor dem App-Import. Einzelne Testpfade wurden an tatsächliche Windows-Fenstergeometrie und Tk-Schriftmetrik angepasst; der App-Code blieb unverändert.

2.5 Historischer Abbruchpunkt und Dokumentprüfung

Die eingebetteten Zeitangaben belegen die Reihenfolge: Source 2.5.1 und Tests waren fertig; die DOCX wurde am 31.08.2026 um 18:21:39 MESZ geändert und das zugehörige 21-seitige PDF 27 Sekunden später erzeugt. Danach fehlt ein abgeschlossener QA-Bericht. Der Lauf endete somit unmittelbar vor der angekündigten visuellen Kontrolle und finalen Übergabe.

Historischer Befund aus der Revision 2.5.2: Die damaligen 21 Seiten wurden auf Inhaltsverlust, Tabellenumbrüche, Navigation und Schriften geprüft. Seitdem enthält die Vorlage wiederholte Tabellenköpfe, interne Sprungziele sowie explizite Arial-/Consolas-Schriften. Die aktuelle Revision verwendet diese Gestaltung weiter.

2.6 Entwicklungsschritte seit 2.6.0

Die ältere Entwicklung bis 2.6.0 bleibt im archivierten Word-Dokument und im CHANGELOG erhalten. Die folgenden Schritte erklären den heutigen Funktionsstand; historische Versionsnummern werden dabei bewusst nicht umetikettiert.

- 2.7 bis 2.9 – Papierkorb, Labels und Kalender; Long-Task und Zwischenüberschrift; verschachtelte Ordner mit Kreis- und Tiefenschutz. Die Datenformate wurden schrittweise bis 9 erweitert.
- 2.10 bis 2.11 – TXT-Rundlauf der vier Punktarten, Labeldarstellung, Papierkorb für einzelne Punkte, Bestandswächter und optionale Uhrzeit. Seit 2.11.0 gilt Datenformat 10.
- 2.12 bis 3.0.2 – kompaktere Eingabemaske, Label-Aufklappfeld, Kalenderknopf, Umbenennen in der Seitenleiste, Symbole sowie Drag & Drop in Gruppen. 3.1.0 vereinheitlicht Änderungen und modale Dialoge; wirkungslose Aktionen kosten keinen Rückgängig-Schritt mehr.
- 3.2.0 – automatische Übernahme aus früheren Programmnamen entfernt, ICONS-Tabelle bereinigt, reproduzierbarer Beispielbestand ergänzt. Datenformat 10 bleibt unverändert. Windows-/macOS-Freigaben und Release-Builds folgen als eigene Arbeit.

Datenformat 10 ergänzt unter anderem Papierkorbeinträge für einzelne Punkte und optionale Uhrzeit. Die aktuelle App kann portable Backups 4 bis 10 lesen. Alte App-Versionen sind kein sicherer Bearbeitungsweg für neue Bestände; für einen Wechsel zuerst ein Komplettbackup erstellen. TXT überträgt keine Anhangsbinarys und ersetzt kein Komplettbackup.

Ein Nebenbefund betrifft die Testumgebung. Der Integrationstest isolierte den Nutzerdatenordner bislang nur über APPDATA, was ausschließlich unter Windows wirkt. Auf einem Mac oder unter Linux hätte ein Testlauf die echten Nutzerdaten verwendet und überschrieben. Seit 2.5.5 überschreibt die Umgebungsvariable GLIDE_DATA_DIR den Datenordner plattformunabhängig; der Test prüft diese Isolierung als erste Zusicherung.

3. Was bereits richtig vorbereitet ist

Bereich	Bewertung
Windows EXE	geplant; Build fehlt
Windows Installer	geplant; Konfiguration fehlt
macOS .app	geplant; Build fehlt
DMG	geplant; Build fehlt
Windows ICO	Anforderungen bekannt; Asset fehlt
macOS ICNS	Anforderungen bekannt; Asset fehlt
Apple Icon Composer	berücksichtigt
PyInstaller OneDir	Planungsoption; Spec fehlt
Windows Signing	offen
Apple Developer ID	offen
Notarisierung	offen
Benutzerdaten außerhalb Programmordner	im Code umgesetzt
Versionierung	3.2.0 konsistent
Updatefähigkeit	Datenmigration vorhanden; Installer ungeprüft
Clean-Machine-Tests	offen
Microsoft Store	Anforderungen recherchiert
Mac App Store	Sandbox und Build offen
KI-Arbeitsauftrag	bereits vorhanden
Release-Checkliste	bereits vorhanden

PyInstaller OneDir ist die vorgesehene Packaging-Basis, noch kein implementierter Buildprozess. Die spätere Build-Konfiguration muss Python und Tk bündeln, Ressourcen auflösen sowie auf beiden Zielplattformen reproduzierbar geprüft werden.

4. Ergänzungen und offene Punkte der bisherigen Vorbereitung

4.1 Microsoft Store: Silent Installation und unveränderliche Installer

Für eine klassische EXE/MSI-Einreichung sollte die vorhandene Anleitung um folgende Store-Anforderungen ergänzt werden:

- Silent Installation ohne Benutzerinteraktion muss funktionieren.
- Der Installer darf kein Webstub/Downloader sein, der wesentliche Bestandteile erst während des Setups nachlädt.
- Download-URLs sollten versioniert und dauerhaft eindeutig sein.
- Eine bereits eingereichte Datei hinter einer Download-URL darf nicht stillschweigend ersetzt werden.
- Installer und relevante PE-Binaries müssen digital signiert sein.
- Returncodes und Silent-Uninstall sollten für einen professionellen Win32-Release definiert und getestet werden.

```
Glide-Setup-<VERSION>.exe /VERYSILENT /SUPPRESSMSGBOXES /NORESTART
```

4.2 Inno Setup: dauerhaft stabile AppId

Die Inno-Setup-AppId sollte einmalig erzeugt und anschließend über alle Glide-Versionen stabil gehalten werden. Sie identifiziert die Installation für Upgrade- und Deinstallationslogik. Eine versehentliche Änderung kann zu parallelen Installationen oder doppelten Deinstallationsinträgen führen.

```
AppId={{EINMALIG-GENERIERTE-STABILE-GUID}}
```

Wichtig ist die Trennung der Identitäten:

- Inno Setup AppId – Installationsidentität
- Windows AppUserModelID – Taskleisten-/Shell-Zuordnung
- macOS Bundle Identifier – Apple-Bundle-Identität
- Produktname – sichtbarer Marken-/Anwendungsname
- Versionsnummer – Release-Version

4.3 Windows Code Signing: SHA-256 und RFC-3161 Timestamping

Für öffentliche Windows-Releases sollte die Signaturstrategie nicht nur „signieren“, sondern auch den Zeitstempel festlegen:

```
SHA-256 Signatur
+
RFC-3161 Timestamp
```

Das Timestamping stellt sicher, dass eine korrekt signierte Veröffentlichung auch nach Ablauf des verwendeten Zertifikats weiterhin als zum Signierzeitpunkt gültig bewertet werden kann. Automatisierung sollte erst erfolgen, wenn der unsigned Build reproduzierbar und stabil funktioniert.

4.4 Apple Notarisierung: finalen Workflow präzisieren

Für den produktiven macOS-Release sollte der Ablauf klar in technische Prüfschritte zerlegt werden:

```

Glide.app bauen
↓
alle eingebetteten Binaries prüfen
↓
Glide.app mit Developer ID signieren
↓
Signatur verifizieren
↓
DMG erzeugen
↓
DMG signieren
↓
finale DMG mit notarytool an Apple senden
↓
Notarisierung prüfen
↓
Ticket stapeln
↓
Gatekeeper-Test

```

Für aktuelle Apple-Workflows ist notarytool der relevante CLI-Weg. Hardened Runtime und korrekte Signierung der eingebetteten Bestandteile sind zentrale Voraussetzungen.

4.5 Marken-/Namensprüfung für „Glide“

Die Namens- und Markenlage zu Glide ist nicht freigegeben. Vor öffentlichem Branding sind Markenregister, Store-Verfügbarkeit und Domains fachlich zu prüfen. Aus einem Namensfund allein folgt keine Aussage über Zulässigkeit oder Rechte.

- DPMA-Recherche
- EUIPO-Recherche
- relevante Waren-/Dienstleistungsklassen für Software
- Store-Namensverfügbarkeit
- Domain-/Webauftritt
- mögliche Verwechslungsgefahr

Hinweis

Die Markenprüfung in diesem Dokument ist eine Projekt- und Risikomanagement-Empfehlung, keine Rechtsberatung.

5. Branding- und Icon-Vorbereitung

Die bisherige Spezifikation mit einem hochwertigen 1024 × 1024 px Master, Vektorquelle und reproduzierbaren Exporten ist richtig. Technische Formate sollten aus einem kontrollierten Master erzeugt werden statt als voneinander unabhängige Masterdateien gepflegt zu werden.

5.1 Empfohlenes Masterpaket

```

Glide_Icon_Master.ai
Glide_Icon_Master.svg
Glide_Icon_Master.psd
Glide_Icon_Master_1024.png

Apple_Layers/
├─ 01_Background.svg
├─ 02_BaseShape.svg
├─ 03_MainSymbol.svg
├─ 04_Foreground.svg
├─ 05_Highlight.svg

```

Für Apple Icon Composer sollten getrennte, sinnvoll benannte Ebenen auf einem 1024 × 1024-Canvas vorbereitet werden. Vektor ist bevorzugt; Text sollte in Konturen umgewandelt werden. Systemmasken und bestimmte systemseitige Material-/Schatteneffekte sollten nicht unnötig in das Masterartwork eingebrannt werden.

5.2 Small-Size-Prüfung

Prüfgröße	Typische Verwendung
1024 px	Master / Store / große Vorschau
256 px	große Windows-Shell-Darstellung

Prüfgröße	Typische Verwendung
128 px	hochauflösende Zwischengröße
64 px	Systemdarstellung
48 px	Explorer / Taskleiste bei Skalierung
32 px	Taskleiste / Start / Titelleiste
24 px	Taskleiste / Suche
16 px	Kontextmenü / kleine Shell-Ansicht

Windows Win32-ICO: mindestens 16, 24, 32, 48 und 256 px gemäß Microsoft. Weitere Größen der Tabelle dienen der optischen Prüfung. Klassisches macOS-iconset: 16, 32, 128, 256 und 512 Punkte jeweils 1x/2x, also bis 1024 px; daraus entsteht .icns. Aktuelle Apple-Gestaltung und Icon Composer separat berücksichtigen. Quellen mit Abruf 04.09.2026: Store-Dokumente.

5.3 Trennung von Win32-ICO und späteren Store-/MSIX-Assets

Für eine klassische Windows-EXE reicht ein korrekt aufgebautes Multi-Resolution-ICO. Ein späteres MSIX-Paket benötigt dagegen ein separates Package-/Store-Assetset. Diese Assets sollten nicht ungeordnet mit den klassischen ICO-Ressourcen vermischt werden.

6. Umgesetzte Gesamt- und Ordnerstruktur

Die Zweiteilung bleibt in 3.2.0 bestehen: persönlicher Workspace für Arbeitsmaterial und ein kanonischer Repository-Bereich. Die folgende Struktur nennt vorhandene zentrale Dateien. Die frühere Modul- und CI-Struktur war ein Zielbild und ist nicht implementiert.

```

01_Repository/Glide/
├── AGENTS.md, README.md, CHANGELOG.md, VERSION
├── LICENSE.md, SECURITY.md
├── src/glide/app.pyw
├── tests/integration/
│   ├── test_glide.py
│   ├── test_datenintegritaet.py
│   └── audit_app.py
├── tests/tools/
│   ├── screenshots.py
│   ├── analyse_statistisch.py
│   ├── analyse_erreichbarkeit.py
│   ├── beispieldaten.py
│   ├── pruefen.py
│   └── releasedaten.py
├── tests/fixtures/
│   ├── current_v4/ bis current_v10/
│   ├── legacy_v2/
│   └── beispiele/
├── docs/
│   ├── 00_INDEX.md
│   ├── 01_PRODUCT_CONSTRAINTS.md
│   ├── 02_ARCHITECTURE.md
│   ├── 05_QA_TESTPLAN.md
│   ├── 06_DATA_BACKUP_MIGRATION.md
│   ├── 07_QA_BERICHT.md
│   ├── 08_CODE_BEFUND.md
│   ├── 09_PROJECT_HANDOFF.md
│   ├── 09_STARTKONTEXT.md
│   ├── 09_ARBEITSAUFTRAG_BESTANDSANALYSE.md
│   ├── 10_RELEASE_CHECKLIST.md
│   ├── 11_BESTANDSANALYSE.md
│   ├── decisions/
│   ├── exec-plans/
│   └── archiv/
├── assets/
├── packaging/
├── scripts/
├── requirements/
└── Historie
    ├── frühere Dokumentstände
    ├── Branding noch zu liefern
    ├── Build-Konfiguration fehlt
    ├── vorbereitete Prozessbereiche
    └── getrennte Laufzeit-/Buildbereiche

```

Keine Modulzerlegung und keine neue CI-Konfiguration beauftragt.
 Finale Binärartefakte gehören nach 30_Release_Exports/<Version>/.
 Historische Dateien bleiben in dezentralen Archiv-Unterordnern.

6.1 Zweck der Zweiteilung

Glide_Workspace ist der persönliche Designer-/Projektarbeitsplatz. Dort dürfen AI-/PSD-Arbeitsdateien, Research, Screenshots, Entwürfe und Archive liegen.

01_Repository/Glide ist die technisch kontrollierte Softwarebasis. Nur dort arbeitet später Codex produktiv. Das reduziert das Risiko, dass alte Python-Kopien, Archive oder Teststände als aktueller Source interpretiert werden.

6.2 Umgang mit der aktuellen Struktur

Aktuell	Empfehlung
__pycache__	generierter Cache, kein Source; historische Funde bleiben archiviert
05_Probelisten_Testdaten	aktuelle 3.2.0-Beispielbackups; ältere TXT-/JSON-Dateien im Archiv
07_Python-Versionen	als Historie beibehalten; kanonischer Source ist src/glide/app.pyw ohne Versionsnummer
10_Dokumentation	umgesetzt: technische Markdown-Doku im Repository; Word-Dokumentation bleibt außerhalb
Dezentrale Archive	im jeweiligen Ordner; 100_Archiv ist kein aktiver Bereich
_test_glide_v250.py	erledigt: kanonischer Integrationstest unter tests/integration/test_glide.py; alte Testkopie archiviert

Wichtig für den nächsten Schritt

Der Monolith bleibt erhalten. Source 3.2.0 und die aktuelle Einzeldatei sind bytegleich. Eine Modulerlegung ist nicht beauftragt; das nächste Arbeitspaket richtet sich auf reale Plattform-QA und reproduzierbares Packaging.

7. Aufgabenverteilung: Was Codex übernehmen kann und was beim Benutzer bleibt

Aufgabe	Codex	Benutzer
Repository analysieren	vollständig	Kontrolle
Import-/Dependency-Analyse	vollständig	–
Source-Struktur planen	vollständig	Freigabe
Dateien verschieben/refactoren	vollständig	Review
.gitignore / .gitattributes / .editorconfig	vollständig	–
AGENTS.md erstellen	vollständig	Regeln freigeben
Dokumentationsstruktur	vollständig	Inhalte freigeben
Tests erstellen	weitgehend	fachliche Prüfung
Smoke Tests	weitgehend	reale Endkontrolle
Datenpfade Windows/macOS	vollständig	–
Migrationen	weitgehend	Backup-/Datenfreigabe
PyInstaller Spec	vollständig	–
Windows Buildskripte	vollständig	ausführen/testen
Inno Setup Script	vollständig	Installer testen
Windows Version Resources	vollständig	Publisherdaten liefern
ICO erzeugen	technisch	Artwork liefern
ICNS erzeugen	technisch	Artwork liefern
Icon-Validator	vollständig	optische Kontrolle
DMG-Skript	vollständig	optische Kontrolle
DMG-Hintergrund technisch einbauen	vollständig	gestalten
Windows Signierskript	vollständig	Zertifikat/Account
Apple Signierskript	vollständig	Developer Account
Notarisierungsskript	vollständig	Credentials/Secrets

Aufgabe	Codex	Benutzer
CI/CD	vollständig	Secrets hinterlegen
SHA-256 Manifest	vollständig	–
Release Notes	vollständig	Freigabe
Store-Texte	sehr gut	Freigabe
Store-Screenshots automatisieren	teilweise	finale Design
Datenschutzerklärung entwerfen	Entwurf	rechtliche Prüfung
Lizenz/EULA entwerfen	Entwurf	rechtliche Entscheidung
Markenprüfung recherchieren	Unterstützung	finale rechtliche Prüfung
Apple Developer Account	nein	Benutzer/Organisation
Microsoft Partner Account	nein	Benutzer/Organisation
Zertifikate anlegen/kaufen	Anleitung	Benutzer/Organisation
Finales Publishing	vorbereiten	Benutzer/Organisation

Rollenverständnis: Codex sollte für Glide primär als Software-/Release-Engineer eingesetzt werden. Das hochwertige Artwork und die Produktentscheidungen bleiben beim Designer bzw. Projektverantwortlichen; Codex übernimmt die technische Ableitung, Validierung, Builds, Tests und Automatisierung.

```

Master-Asset
↓
Validierung
↓
Windows ICO / macOS ICNS / Store Assets
↓
Build
↓
Installer / DMG
↓
Signing
↓
Tests
↓
Release Manifest

```

8. Empfohlene Codex-Arbeitspakete

C01–C08 bleiben wiederverwendbare Ticketvorlagen. Sie sind Dokumentinhalt und führen keine Arbeit automatisch aus. C01 ist in docs/11_BESTANDSANALYSE.md für 3.2.0 nachgeführt; C02/C03 sind teilweise umgesetzt. Asset-, Build- und Signing-Arbeiten benötigen die jeweiligen Produktentscheidungen und Arbeitsaufträge.

C01 – Repository Audit (Stand 3.2.0 dokumentiert)

Analysiere das vollständige Glide-Projekt ausschließlich lesend.

Nimm keine Änderungen am Sourcecode oder an Dateien vor.

Erstelle:

1. vollständige Projektstruktur
2. Liste aller produktiv verwendeten Python-Dateien
3. Import- und Dependency-Analyse
4. Liste aller Runtime-Assets
5. Liste aller hartcodierten Dateipfade
6. Speicherorte der Benutzerdaten
7. mögliche Packaging-Probleme
8. mögliche macOS-/Windows-Inkompatibilitäten
9. Liste alter, redundanter oder möglicherweise archivierter Dateien
10. Risikoanalyse einer späteren Repository-Umstrukturierung

Unterscheide ausdrücklich zwischen produktivem Sourcecode, Testcode, Testdaten, Dokumentation, generierten Dateien und Archivmaterial.

Nimm keinerlei Refactoring vor.

C02 – Repository Foundation (Grundlage umgesetzt)

Erstelle auf Basis des freigegebenen Audit-Berichts eine professionelle Repository-Struktur für Glide.

Ziele:

- konventionelle Python-Projektstruktur
- keine funktionalen Änderungen
- keine Änderung des Datenformats
- keine Änderung der Oberfläche
- keine neuen Runtime-Abhängigkeiten
- alle bisherigen Funktionen müssen weiter funktionieren

Erstelle zusätzlich:

```
.gitignore
.gitattributes
.editorconfig
AGENTS.md
README.md
VERSION
docs/00_INDEX.md
```

Dokumentiere jede Verschiebung und führe anschließend bestehende Tests bzw. Smoke Tests aus.

C03 – Release-Härtung (teilweise umgesetzt)

Prüfe Glide auf Release-Tauglichkeit.

Implementiere bzw. dokumentiere:

- plattformgerechte Datenpfade
- robuste Resource-Pfade für Source- und PyInstaller-Modus
- zentrale Versionsverwaltung
- Datenformat-/Schema-Version
- Backup vor Migrationen
- Logging
- Fehlerbehandlung
- Mehrfachstartverhalten
- Upgrade-Kompatibilität

Keine neuen Produktfunktionen hinzufügen.

C04 – Asset Pipeline (nicht begonnen)

Erstelle für Glide eine reproduzierbare Asset-Pipeline.

Input:

```
assets/branding/master/
```

Output Windows:

- Glide.ico

Output macOS:

- Glide.icns

Store-Assets:

- getrennte Exportordner

Prüfe Abmessungen, Transparenz, Farbraum soweit automatisierbar, Dateinamen und benötigte Auflösungen.

Verändere das Artwork nicht gestalterisch.

C05 – Windows Build (nicht begonnen)

Erstelle einen reproduzierbaren Windows-Releaseprozess für Glide.

Ziel:

Source → PyInstaller OneDir → Glide.exe → Inno Setup → Glide-Setup-x.y.z.exe

Berücksichtige:

- Glide.ico
- GUI-Fenstericon
- AppUserModelID
- Windows Version Resources
- stabile Inno-AppId
- Upgrade vorhandener Versionen
- Deinstallation
- Nutzerdatenerhalt
- Silent Install
- Silent Uninstall
- Logging
- Release-Dateinamen

C06 – macOS Build (nicht begonnen)

Erstelle einen reproduzierbaren macOS-Releaseprozess.

Source → PyInstaller → Glide.app → Codesign-ready App Bundle → Glide.dmg

Berücksichtige:

- Glide.icns
- Bundle Identifier
- CFBundleVersion
- CFBundleShortVersionString
- Mindest-macOS-Version
- arm64/x86_64 Entscheidung
- Hardened Runtime
- Entitlements
- Developer-ID-Signing
- notarytool
- Stapling
- Gatekeeper-Verifikation

C07 – QA (drei Suiten vorhanden; Release-QA offen)

Erstelle und automatisiere den Glide Release-Testplan.

Testbereiche:

- Unit
- Integration
- Smoke
- Packaging
- Installation
- Update
- Datenmigration
- Backup
- Restore
- Deinstallation
- Reinstallation
- Start nach Neustart
- fehlende Assets
- Windows DPI Scaling
- macOS Gatekeeper
- Signaturprüfung

Erzeuge einen maschinenlesbaren Testreport.

C08 – Release Automation (nicht begonnen)

Automatisiere nach erfolgreicher Stabilisierung den Glide Release-Prozess.

Erzeuge:

- Buildskripte Windows
- Buildskripte macOS
- Assetgenerierung
- Testpipeline
- Versionsprüfung
- Release-Manifest
- SHA-256-Hashes
- Release Notes
- Changelog-Unterstützung

Signing Secrets dürfen niemals im Repository gespeichert werden.

9. Eingearbeitete AGENTS.md für Glide

Verbindlich ist 01_Repository/Glide/AGENTS.md. Der folgende Auszug fasst die aktuellen Regeln zusammen. Er ersetzt die Quelldatei nicht und enthält keine zusätzlichen Arbeitsaufträge.

Glide – Arbeitsregeln für den aktuellen Bestand

Produkt: lokale Desktop-App, ohne Konto, Cloud oder Telemetrie.
 Kanonischer Source: src/glide/app.pyw; Version 3.2.0, Schema 10.

Verbindliche Dokumente

- docs/01_PRODUCT_CONSTRAINTS.md
- docs/02_ARCHITECTURE.md
- docs/05_QA_TESTPLAN.md
- docs/06_DATA_BACKUP_MIGRATION.md
- docs/07_QA_BERICHT.md
- docs/10_RELEASE_CHECKLIST.md
- docs/decisions/PRODUCT_IDENTITY.md

Entwicklungsregeln

1. Vor Änderungen aktuellen Teststand und Datenpfade prüfen.
2. GLIDE_DATA_DIR vor jedem Testimport auf isolierten Ordner setzen.
3. Bestehende Funktionen, Nutzerdaten und Migrationen bewahren.
4. Keine neue Laufzeitabhängigkeit ohne dokumentierte Entscheidung.
5. Keine Datenformatänderung ohne eigene Migration, Sicherung und Tests.
6. Punkte über item_change, Listen und Ordner über sidebar_change ändern.
7. Umbauaktionen zusätzlich unter guarded_structural_change ausführen.
8. Modale Dialoge ausschließlich über run_modal führen.
9. Oberflächensymbole in ICONS zentralisieren; Rest-Ausnahmen dokumentieren.
10. Überholte Dokumente vor dem Ändern versionsbezogen archivieren.
11. Änderungen in CHANGELOG und betroffenen Dokumenten nachführen.
12. Keine Secrets, Zertifikate oder Schlüssel ins Repository.

Abschluss

Syntaxprüfung und passende automatisierte Tests müssen bestehen.
 Versionen müssen übereinstimmen.
 Verbleibende manuelle Prüfungen ausdrücklich nennen.

Aktueller Auftrag

Keine Datei löschen. Keine automatische Versionserhöhung oder Commits.
 Den Monolithen und die drei linearen Testsuiten nicht umstrukturieren.
 Inhaberentscheidungen und vermutete Fehler nicht als erledigt ausgeben.

10. Eingerichtetes Dokumentationssystem

Das Markdown-Grundgerüst ist unter 01_Repository/Glide/docs/ eingerichtet. Diese Word-Datei bleibt die ausführliche menschlich lesbare Arbeitsgrundlage; die Markdown-Dateien bilden die knappen code-nahen Quellen. Noch nicht angelegte Spezialdokumente folgen erst zusammen mit Packaging, Signing und Store-Arbeit.

Datei	Zweck
00_INDEX.md	Navigation und Dokumentationsindex
01_PRODUCT_CONSTRAINTS.md	Produktgrenzen und bewusste Nicht-Ziele
02_ARCHITECTURE.md	Monolith, Datenmodell und Datenfluss
07_QA_BERICHT.md	aktueller Laufstand und verbleibende Prüfungen
08_CODE_BEFUND.md	statische Befunde und Maßnahmen
05_QA_TESTPLAN.md	vollständiger Testplan
06_DATA_BACKUP_MIGRATION.md	Speicherorte, Schema, Backup, Restore
09_PROJECT_HANDOFF.md	Einstieg, Architektur und Übergabe
09_STARTKONTEXT.md	kompakter Startkontext
11_BESTANDSANALYSE.md	Befund, Maßnahmen, Prozesse und Grenzen
10_RELEASE_CHECKLIST.md	Go/No-Go-Checkliste
CHANGELOG.md	Änderungshistorie
SECURITY.md	Sicherheitsprinzipien
AGENTS.md	Codex-Navigation und dauerhaft geltende Regeln

10.1 PRODUCT_CONSTRAINTS.md als Schutz vor KI-Fehlentscheidungen

Diese Datei verhindert, dass Codex technisch plausible, aber strategisch unerwünschte Entscheidungen automatisch einführt – etwa SQLAlchemy, platformdirs, Cloudservices, Telemetrie oder ein Kontensystem.

```
# Produktgrenzen

Glide ist eine lokale Desktop-App für Windows und macOS.
Deutsch ist die einzige Oberflächensprache.
Die Kernfunktionen benötigen kein Internet und kein Benutzerkonto.
Glide selbst überträgt keine Nutzerdaten.

Laufzeit: Python-Standardbibliothek und Tk.
Neue Laufzeitabhängigkeiten benötigen eine dokumentierte Entscheidung.
Buildtools zählen nicht zur Laufzeit der Anwendung.

Bewusste Nicht-Ziele
- Cloud-Synchronisierung und Mehrbenutzerbetrieb
- Benutzerkonten, Lizenzserver und Telemetrie
- Push-Infrastruktur und externe Kalender-/Mailanbindung
- Mehrsprachigkeit und Rich-Text-Editor

Datenkompatibilität hat Vorrang vor internem Refactoring.
Maßgeblich ist docs/01_PRODUCT_CONSTRAINTS.md.
```

11. Zentrales Produkt- und Identitätsregister

Die verbindlichen Stammdaten werden zentral im Repository geführt. Die externe Liste Offene_Entscheidungen_3.2.0.md ist eine Arbeitsansicht desselben Registers, kein zweiter Ort für verbindliche Identitäten.

```
01_Repository/Glide/docs/decisions/PRODUCT_IDENTITY.md

Produktname: Glide
Langname: Glide – Aufgaben und Listen
App-Version: 3.2.0
Datenformat: 10
Lesbare portable Backups: 4 bis 10
Status: interne Vorabversion, kein öffentliches Release

Offen: Publisher, Copyright, Supportkontakt, Website,
Datenschutz-URL, Lizenzmodell und Preis.
Windows: Architektur, Mindestversion, AppUserModelID, AppId.
macOS: Bundle Identifier, Mindestversion, Architektur, Vertriebsweg.
Kennungen erst nach Inhaberentscheidung dauerhaft festlegen.
```

12. Apple- und Microsoft-Accountentscheidungen früh treffen

12.1 Apple Developer Program

Vor öffentlichem Vertrieb sollte entschieden werden, ob als Einzelperson oder Organisation veröffentlicht wird. Bei einer Organisation sind Unternehmensdaten und je nach Apple-Prozess unter anderem eine D-U-N-S-Nummer sowie ein geeigneter geschäftlicher Web-/E-Mail-Auftritt relevant. Der veröffentlichte Verkäufer-/Entwicklername hängt von der gewählten Kontostruktur ab.

Diese Entscheidung sollte vor finaler Store-Gestaltung und vor dem Festlegen dauerhaft verwendeter Identitäten getroffen werden.

12.2 Microsoft Partner Center

Auch Microsoft unterscheidet Individual- und Company-Konten. Die Kontostruktur sollte frühzeitig festgelegt werden, da Publisher-Identität, Verifikation und spätere Store-Verwaltung davon abhängen. Account- und Identitätsentscheidungen sind nicht an Codex delegierbar.

13. QA: vom Checklistenpunkt zum reproduzierbaren Testsystem

Drei lineare Python-Suiten prüfen Glide mit isolierten temporären Daten. Hauptsuite, Datenintegritätssuite und appweiter Audit bleiben einzeln aufrufbar. Das Prüfwerkzeug `tests/tools/pruefen.py` bündelt Syntax, Versionen, Suiten und Analysen. Der aktuelle QA-Bericht dokumentiert konkrete Laufzeit, Ergebnisse und übersprungene Schritte; manuelle Plattform- und Buildtests bleiben separat.

```
tests/
├── integration/test_glide.py
├── integration/test_datenintegritaet.py
├── integration/audit_app.py
├── tools/pruefen.py           Schnell- und Vollprüfung
├── tools/screenshots.py      visuelle Nachweise
├── tools/analyse_statistisch.py
├── tools/analyse_erreichbarkeit.py
└── fixtures/                 Schema 4-10, Legacy 2, Beispiele
```

Nachweis 04.09.2026: Windows-Prüfung mit Python 3.12.12 und Tk 8.6.17. Die drei bestehenden Suiten bestanden nach begrenzten Anpassungen im Testcode für aktuelle Widgets und native Geometrie. App-Code und Datenformat blieben unverändert. Endgültiger Gesamtlauf einschließlich Release-Fixture: siehe `docs/07_QA_BERICHT.md`.

Ebene	Ziel
Unit	Einzelne Logikbausteine isoliert prüfen.
Integration	Zusammenspiel von Daten, Speicherung, Laden und Modulen prüfen.
Smoke	App starten und wichtigste Kernfunktionen in kurzer Sequenz prüfen.
Packaging	Tatsächlich gebaute EXE, .app, Installer bzw. DMG testen.

13.1 Release-Testmatrix

Test	Windows – Soll	macOS – Soll
Source-Start	erforderlich	erforderlich
PyInstaller Build	erforderlich	erforderlich
Clean Install	erforderlich	erforderlich
Erster Start	erforderlich	erforderlich
Daten anlegen	erforderlich	erforderlich
Neustart	erforderlich	erforderlich
Daten vorhanden	erforderlich	erforderlich
Update Alt → Neu	erforderlich	erforderlich
Backup	erforderlich	erforderlich
Restore	erforderlich	erforderlich
Uninstall	erforderlich	nicht anwendbar
Nutzerdaten erhalten	erforderlich	erforderlich
Reinstall	erforderlich	erforderlich
Icon	erforderlich	erforderlich
DPI/Scaling	erforderlich	erforderlich (Retina)
Signatur	erforderlich	erforderlich
Offline	erforderlich	erforderlich
Clean Machine ohne Python	erforderlich	erforderlich

14. Referenz-Testdaten und Migration

Referenzdaten liegen für die Formate 4 bis 10 sowie Legacy 2 vor und werden vom Integrationstest geladen und normalisiert. Das Beispiel-Komplettbackup und die Release-Arbeitslisten sind über Werkzeuge reproduzierbar. Ein Komplettbackup ersetzt beim Laden den gesamten Bestand; ein TXT-Listenimport ergänzt eine einzelne Liste.

```
tests/fixtures/
├── current_v4/ bis current_v10/   ein Referenzbestand je Schema
├── legacy_v2/                     historischer Aufgabenbestand
└── beispiele/                     portable .glidebackup-Beispiele

tests/tools/beispieldaten.py       erzeugt Funktionsbeispiele
tests/tools/releasedaten.py        erzeugt drei Release-Arbeitslisten
90_Testdaten_Extern/              unveränderte historische TXT-Eingaben
```

Format 10 umfasst vier Punktarten, Labels, verschachtelte Ordner, Papierkorb und Uhrzeit. Zwei isoliert belegte Release-Blocker bleiben offen: Ein Systemlabel kann die Grenze auf 21 Labels überschreiten; make_subitem kann Tiefe 101 erzeugen, die später beim Laden abgewiesen wird. App-Code unverändert; gezielte Behebung und Regression erforderlich. Alte Fixtures bleiben historische Schemareferenzen.

15. Release-Artefakte und Versionierung

Finale Veröffentlichungen gehören nach 30_Release_Exports/<VERSION>/. Für Source 3.2.0 existiert noch kein freigegebener EXE-, Installer-, App- oder DMG-Build. Ältere Ablagepläne bleiben historische Vorbereitung. Das folgende Ablageschema ist ein Zielbild, kein vorhandenes Release-Manifest.

```
30_Release_Exports/
├── <VERSION>/
│   ├── Windows/Glide-Setup-<VERSION>-<ARCH>.exe
│   ├── macOS/Glide-<VERSION>-<ARCH>.dmg
│   ├── RELEASE_NOTES.md
│   ├── release-manifest.json
│   └── Hashes/SHA256SUMS.txt
└── {
    "product": "Glide",
    "version": "<VERSION>",
    "channel": "<CHANNEL>",
    "build_date": "<YYYY-MM-DD>",
    "windows": {"architecture": "<ARCH>"},
    "macos": {"architecture": "<ARCH>"}
}
```

Ein final freigegebenes Release-Artefakt sollte danach nicht mehr unter identischem Dateinamen ausgetauscht werden. Korrekturen erhalten eine neue Version bzw. Buildnummer.

16. Security und Software-Supply-Chain

```
Pinned Build-Dependencies
↓
Clean Build Environment
↓
Build
↓
Malware Scan
↓
Signing
↓
Signature Verification
↓
SHA-256
↓
Release Manifest
```

Besonders wichtig ist die Trennung von Build-Konfiguration und geheimen Signierdaten. Folgende Dateien bzw. Geheimnisse dürfen niemals ins Repository:

- *.pfx
- *.p12
- *.key
- *.pem
- Apple API Keys

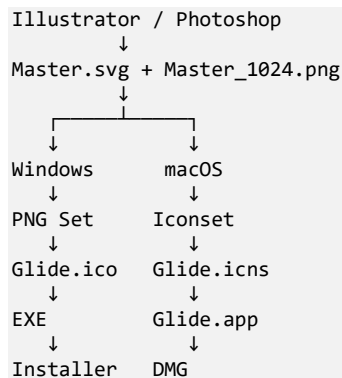
- Notary Credentials
- Signing Tokens
- Passwörter und Recovery-Codes

Repository-Regel

Signing-Secrets gehören in Betriebssystem-Keychains, Secret Stores oder CI-Secrets. Das Repository enthält ausschließlich Skripte und Konfiguration, niemals private Schlüssel.

17. Empfohlener Grafik- und Asset-Workflow

Die grafische Arbeit bleibt in Illustrator/Photoshop. Die technische Größen- und Formatvielfalt wird anschließend reproduzierbar erzeugt.



Das Artwork wird nicht durch Codex „verbessert“ oder gestalterisch verändert. Codex validiert Größen, Dateiformate und Exportvollständigkeit und automatisiert die Ableitung technischer Assets.

18. Nachgeholte Arbeitsvorbereitung und verbleibende Punkte

18.1 Priorität 1 – funktionierenden Stand einfrieren

Der Source 3.2.0 ist im kanonischen Repository-Pfad vorhanden und bytgleich zur aktuellen Einzeldatei unter 07_Python-Versionen. Ältere Source-Fassungen, Word-Dokumente und Checklisten bleiben unverändert archiviert. Die Archivzuordnung dieser Revision steht in der Bestandsanalyse.

Glide 3.2.0 · Source-SHA-256: 6309f2669f549077...2e3cd4bfc01715

Der Monolith wurde nicht in Module zerlegt; die Strukturänderung beschränkt sich auf eine getestete kanonische Kopie und begleitende Projektdateien.

18.2 Priorität 2 – Produktidentität festlegen

- Publisher
- Copyright
- Supportkontakt
- Website
- Bundle Identifier
- Windows AppUserModelID
- Inno Setup AppId
- macOS Mindestversion
- macOS Architektur
- Windows Architektur
- Lizenzmodell
- Release-Kanäle
- Datenformat 10 ist bereits festgelegt; keine offene Geschäftsentscheidung

18.3 Priorität 3 – Branding-Master vorbereiten

```
Glide_Icon_Master.ai
Glide_Icon_Master.svg
Glide_Icon_Master.psd
Glide_Icon_Master_1024.png

Apple-Layer:
01_Background.svg
02_Base.svg
03_Symbol.svg
04_Foreground.svg
```

Anschließend optische Tests in 1024, 256, 128, 64, 48, 32, 24 und 16 px durchführen.

18.4 Priorität 4 – Referenz-Testdaten definieren

Vorhandene Referenzen decken alte Schemata und typische Grenzfälle ab. Beispiele werden aus lesbarem Quelltext erzeugt und als .glidebackup validiert. Bei Änderung des Erzeugers die Datei neu bauen und testen; Fälligkeiten werden am Erzeugungstag berechnet.

18.5 Priorität 5 – Dokumentationsgerüst anlegen

AGENTS.md und docs/00_INDEX.md	verbindliche Einstiege
docs/02_ARCHITECTURE.md	Version 3.2.0 / Schema 10
docs/05_QA_TESTPLAN.md	Prüfprozess
docs/06_DATA_BACKUP_MIGRATION.md	Formate, Speicherung, Import
docs/07_QA_BERICHT.md	aktueller Nachweis
docs/08_CODE_BEFUND.md	bekannte Befunde
docs/09_PROJECT_HANDOFF.md	Übergabe
docs/10_RELEASE_CHECKLIST.md	offene Release-Gates
docs/11_BESTANDSANALYSE.md	Konsolidierung und Ergebnisse
docs/decisions/PRODUCT_IDENTITY.md	Identitäten und offene Entscheidungen
docs/exec-plans/ und docs/archiv/	unveränderte Historie

18.6 Priorität 6 – Store-Grafiken noch nicht final produzieren

Store-Screenshots erst vom Release-Kandidaten auf dem jeweiligen Zielsystem erstellen. Mac: 1–10 PNG/JPEG ohne Alpha, 1280×800, 1440×900, 2560×1600 oder 2880×1800. Microsoft MSI/EXE: 1–10, mindestens vier empfohlen; Pixelvorgaben dieser Route sind in der aktuellen Quelle nicht genannt. Quellen und Abrufdatum 04.09.2026 in 40_Store_Material.

19. Verbindlicher Phasenplan

Phase	Verantwortung	Ergebnis
Phase 0 – Arbeitsvorbereitung	Benutzer	Source 3.2.0, Beispiele und Dokumentation vorhanden; Identität und Branding offen
Phase 1 – Audit	Codex	Bestandsanalyse 3.2.0 vorhanden; Grenzen und Nachtests getrennt dokumentiert
Phase 2 – Repository-Härtung	Codex	Struktur, Regeln, Doku und drei Suiten vorhanden; keine Modulzerlegung beauftragt
Phase 3 – Asset-Pipeline	Benutzer + Codex	nicht begonnen: freigegebenes Artwork fehlt
Phase 4 – Windows Release Candidate	Codex + Benutzer	nicht begonnen: Packaging-/Installer-Konfiguration fehlt
Phase 5 – macOS Release Candidate	Codex + Benutzer	nicht begonnen: macOS-Buildumgebung und Identitäten fehlen
Phase 6 – QA	Codex + Benutzer	automatisierte Windows-Prüfung vorhanden; Geräte-, DPI-, macOS- und Clean-Machine-Freigabe offen
Phase 7 – Signing	Benutzer + Codex	Windows Authenticode, Apple Developer ID, Notarisierung; Secrets außerhalb Repository
Phase 8 – Direct Release	Benutzer	signierte EXE/DMG direkt veröffentlichen; reale Nutzer-/Clean-Machine-Erfahrung sammeln
Phase 9 – Stores	Benutzer + Codex	Vertriebsweg durch Inhaber festlegen; Store-Vorgaben und Sandbox anhand des Builds prüfen
Phase 10 – Automatisierung	Codex	Prüfprozess gebündelt; Build, Signing und Veröffentlichung noch nicht automatisiert

20. Strategische Leitprinzipien

Automatisierungsprinzip

Funktioniert manuell → ist dokumentiert → ist reproduzierbar → ist getestet → wird automatisiert.

Die umgekehrte Reihenfolge – zuerst komplexe Automatisierung zu bauen und anschließend zu hoffen, dass der technische Prozess stimmt – ist insbesondere bei PyInstaller, Installer, Signierung, Notarisierung und Datenmigration unnötig riskant.

Zweites Leitprinzip: Keine strukturelle Änderung ohne reproduzierbaren Ausgangstest. Vor dem Verschieben oder Refactoring produktiver Dateien soll Codex zunächst die tatsächlichen Imports, Pfade, Assets und Datenzugriffe des aktuellen Projekts analysieren.

Drittes Leitprinzip: Feature-Entwicklung und Release-Härtung getrennt behandeln. Neue Produktfunktionen dürfen nicht stillschweigend in Packaging-/Build-Tickets hineinrutschen.

21. Zielzustand für den späteren Release-Workflow

```

Benutzer:
Version und Design freigeben
↓
Codex:
Code analysieren
Tests ausführen
Version prüfen/setzen
Assets validieren
↓
Windows-Workflow:
EXE → Installer → Tests
↓
macOS-Workflow:
.app → DMG → Tests
↓
Release-Workflow:
Hashes → Manifest → Release Notes
↓
Benutzer:
Finale Freigabe
↓
Signing / Veröffentlichung

```

Dieser Zielzustand reduziert manuelle Wiederholungsarbeit, ohne die finale fachliche, gestalterische, rechtliche oder accountgebundene Freigabe aus der Hand zu geben.

22. Fazit und nächste konkrete Schritte

Glide 3.2.0 ist als Source und Dokumentation konsolidiert. Die App enthält vier Punktarten, Labels, Papierkorb, Kalender und Datenformat 10. Drei automatisierte Suites und reproduzierbare Beispiele unterstützen die Prüfung. Ein öffentlicher Release ist weiterhin von Plattformfreigabe, Produktidentität, Branding, Packaging und Signing abhängig.

Empfohlene Reihenfolge für die nächsten Arbeiten:

1. Den Stand 3.2.0 mit einer Kopie realer Daten in GLIDE_DATA_DIR prüfen; Originaldaten unverändert lassen.
2. Windows-/macOS-Dialoge, Mehrfachauswahl, DPI, Symbole und die gemeldete Einfrierstörung gezielt manuell prüfen.
3. Produktidentität, Lizenz, Supportkontakt und Plattform-IDs verbindlich festlegen.
4. Branding-Master und Apple-Layer liefern und validieren.
5. PyInstaller-OneDir-Build reproduzierbar und zunächst unsigned erstellen.
6. Windows-Installer und macOS-App/DMG auf Clean Machines testen.
7. Erst danach Signing, Notarisierung, Hashes und Release-Manifest umsetzen.
8. Store-Material erst auf Basis eines nahezu eingefrorenen Release Candidates erstellen.
9. Den Bestandswächter und Rückgängig-Rahmen bei weiteren Änderungen beibehalten; keine unbeauftragte Modulzerlegung.

Nächster sinnvoller Arbeitsauftrag

Nach der Bestandsanalyse zuerst die manuelle Plattformprüfung durchführen und Produktidentitäten festlegen. Danach einen reproduzierbaren unsigned Build samt Smoke-Test erstellen. Signing, Stores und Veröffentlichung bleiben weitere Arbeitsschritte mit eigenen Nachweisen.

23. Quellen und Prüfstand

Interne, in dieser Revision tatsächlich geprüfte Projektgrundlagen:

- Arbeitskontext und Bestandsanalyse-Auftrag vom 04.09.2026; historische Screenshots und Tickettexte bleiben als solche erkennbar.
- Kanonischer Source `src/glide/app.pyw`, Version 3.2.0 und Datenformat 10; frühere Releases anhand des unveränderten CHANGELOG und der Archive.
- Lokaler Workspace und drei Testsuiten unter `tests/integration`; Prüfprozess und abschließende Ergebnisse in `docs/07_QA_BERICHT.md` und `docs/11_BESTANDSANALYSE.md`.

- Technische Markdown-Dokumentation unter 01_Repository/Glide/docs/ und Versions-/Änderungsdateien im Repository-Root.
- Hinweis: Im Workspace lagen keine eigenständige Build-/Release-Anleitung und kein separater Funktionskatalog als lokale Datei vor; Aussagen dazu aus der Vorversion sind daher Planungsbezug, kein in dieser Revision reproduzierter Buildnachweis.

Externe Referenzen aus der Vorlage bleiben erhalten. Aktuell recherchierte Store-Spezifikationen mit Abruf 04.09.2026 stehen in 40_Store_Material/Microsoft/Store_Angaben_Microsoft.md und Apple/Store_Angaben_Apple.md. Die nachfolgenden allgemeinen Links sind historische Quellen, keine Freigaben.

- [OpenAI – Introducing the Codex app](#)
- [OpenAI – Introducing Codex](#)
- [OpenAI – Harness engineering](#)
- [PyInstaller – Operating Mode](#)
- [Microsoft Learn – App package requirements for MSI/EXE apps](#)
- [Microsoft Learn – App icon construction](#)
- [Inno Setup Help – Appld](#)
- [Microsoft Learn – Artifact Signing certificate management](#)
- [Microsoft Learn – Artifact Signing integrations](#)
- [Apple Developer – Notarizing macOS software before distribution](#)
- [Apple Developer – Icon Composer](#)
- [Apple Developer – Program enrollment](#)
- [Apple Developer – Program membership/enrollment](#)
- [Apple Developer – App Store Connect screenshot specifications](#)
- [Apple Developer – macOS distribution](#)
- [Microsoft Learn – Open a developer account](#)
- [Glide – vorhandenes Softwareangebot zur Namens-/Markenrisikoprüfung](#)

Prüfstand 04.09.2026: App 3.2.0, Datenformat 10, portable Backups 4 bis 10. Der aktuelle Gesamtlauf ist im QA-Bericht belegt. Manuelle Plattformprüfung, Einfrier-Reproduktion, Installer, Signing, Store-Einreichung und Clean-Machine-Prüfung bleiben offen. Die ältere Dokumentfassung 2.6.0 ist unverändert archiviert.